NOSH ROBOTICS CV INTERN TASK

Egg Segmentation and Individual Egg Tracking

Computer Vision Intern Task: SAM reference masks, classical image processing, evaluation and video tracking

Name: Yasmeen

Introduction

This report covers the CV intern assignment on egg segmentation. The task has four parts. First, a pretrained model (SAM) is used to segment the eggs in every image of a small dataset. Second, I build my own segmentation method with classical image processing only, split the data into development and evaluation sets, and keep a log of every experiment. Third, I evaluate the classical method against the SAM masks with segmentation metrics and look closely at where it fails. Fourth, I run the method on a video and track each egg separately.

Egg segmentation looks easy but is awkward for classical methods. Eggs are smooth and have almost no texture, so there is nothing to grab onto except colour and outline. Many photos show a white or pale egg on pale cloth, plastic or a steel pot, so the egg and its background are close in colour. Brown eggs have the same colour range as skin, which makes hands a constant source of wrong detections. Shadows and reflections create bright or dark blobs that are also roughly oval. Some images have several eggs, sometimes touching, and in the video a hand hides the eggs for stretches of time.

My approach was this. I used SAM to create reference masks and cleaned them. I split the images into development and evaluation sets by groups, so that near-duplicates cannot sit on both sides. For the classical method I generate many candidate regions by thresholding several colour channels, and then pick the candidates that look most like an egg (ellipse-like, convex, with a clear edge). Every design decision was tested on the development set only and written into the experiment log. The final method was then evaluated on the held-out set and extended to the video with a small tracker.

Short summary of the outcome. On the held-out set (19 images) the final classical method reaches a mean IoU of 0.437 against the SAM masks, and 0.481 on the development set (87 images). Only roughly one image in four (27.6% on the development set, 21.1% on the held-out set) reaches IoU 0.8. The method either works well or fails completely, mostly on pale eggs on pale backgrounds. The video tracking works on the demo video, but it needed one manual identity anchor at a moment where a hand hides both identical eggs at once. I describe both the results and the failures in detail below.

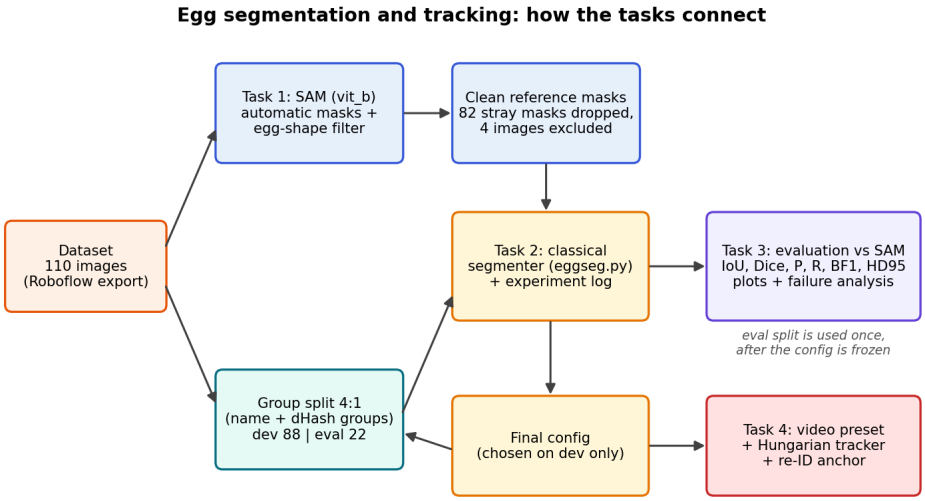


Figure 1: Overall workflow. The evaluation set is only used after the configuration has been fixed on the development set.

Task 1: Segmentation Using SAM

Model and how I ran it

I used the Segment Anything Model (SAM) from Meta AI with the ViT-B checkpoint (sam_vit_b_01ec64.pth, about 375 MB). I also downloaded the larger ViT-H checkpoint, but used ViT-B for all masks because ViT-H is slow on a CPU. The model is loaded with sam_model_registry and runs through SamAutomaticMaskGenerator, which proposes masks for the whole image without any prompt.

The dataset has no bounding boxes, so I could not give SAM a box prompt per egg. Instead I used the automatic generator with points_per_side=24, pred_iou_thresh=0.86, stability_score_thresh=0.90 and min_mask_region_area=200, and then filtered the proposals myself.

Choosing the egg masks

SAM returns many masks per image (eggs, hands, backgrounds, objects). I keep a mask only if it looks like an egg:

- its area is between 1% and 75% of the image;
- more than half of its pixels are not skin-coloured (YCrCb skin range), so hands are rejected;
- its score, which is the IoU between the mask and its best-fit ellipse multiplied by its convexity, is high. The best mask is always kept. Further masks are only kept if this score is at least 0.85 (this allows several eggs), they do not overlap an already kept mask by more than 0.5 IoU, and at most 6 masks are kept.

For every image the script saves a binary mask (union of eggs), an instance label map, and an overlay. I also build contact sheets (48 thumbnails each) so that I can look through all results quickly.

What went wrong with SAM, and how I cleaned it

Going through the contact sheets, most masks were fine (Figure 2), but a clear number were not (Figure 3). The typical problems were: a pale egg in water that was not found at all, a mask that covered the whole pot instead of the egg, ragged masks on an egg in grass, masks on a red stick in an egg tray, and extra masks on wall tiles or on the corner of the image.

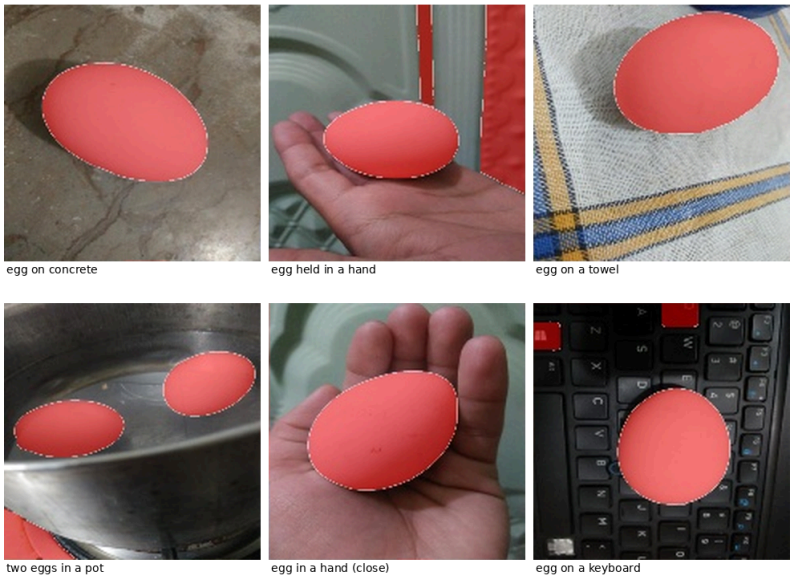


Figure 2: Examples where SAM gave a good egg mask (red overlay, thumbnails from my contact sheets, before cleaning).

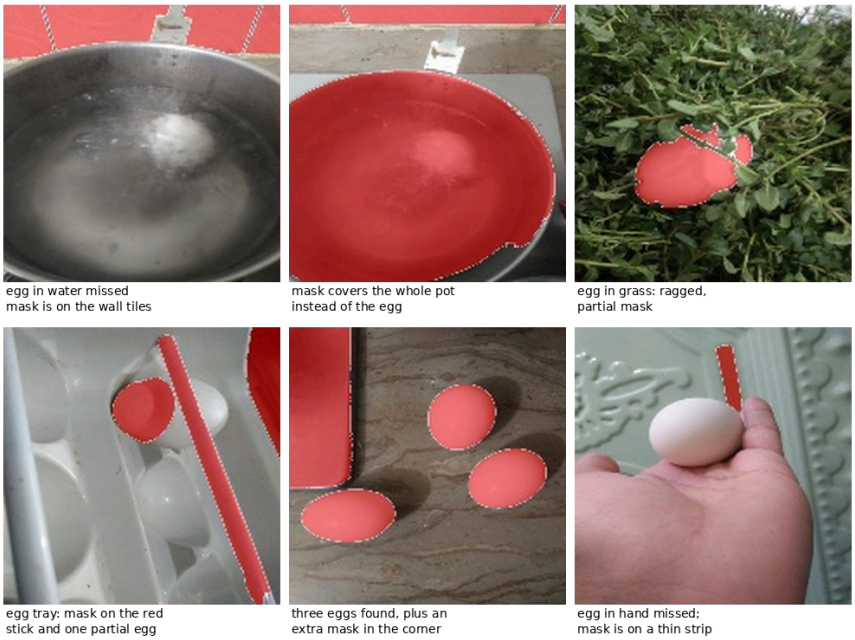


Figure 3: Examples of imperfect SAM masks (thumbnails from my contact sheets, before cleaning).

Many of the extra masks were slices of wall tiles or carpet cut off by the image border. They passed my ellipse filter because a slice of an ellipse at the border still fits an ellipse well. I wrote a second cleaning step (clean_gt.py) that removes a mask if more than 15% of its outline lies on the image border, or if it covers less than 0.4% of the image, but only when another mask remains in that image. It removed **82 masks** in total. After that I excluded four images completely (image numbers 39, 100, 135 and 145) because the SAM mask was not on the egg.

This cleaning changed the evaluation a lot. My first evaluation, run before cleaning, looked much worse than it should have, because in several images my method found the egg correctly but the SAM reference was on the wall (Figure 4). Table 1 shows the effect.

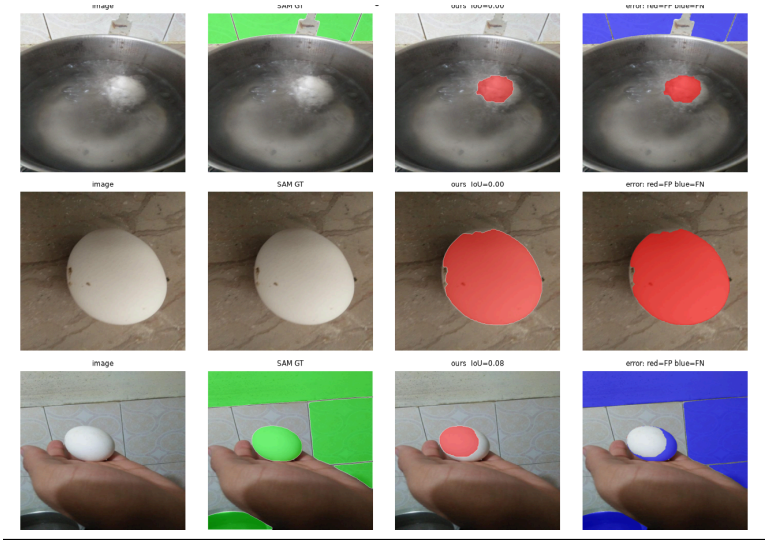


Figure 4: Reference-mask errors found in my first evaluation (before cleaning). Columns: image, SAM reference (green), my result (red), error map. Top row: the reference is the wall, my mask is on the egg in the water. Middle row: the reference mask is empty. Bottom row: the reference includes wall tiles.

Table 1: Effect of cleaning the SAM reference masks (same method, before and after cleaning).

Split	Images	Mean IoU	Median IoU	Count agr.	Comment
Eval, before cleaning	22	0.334	0.231	0.59	many wrong reference masks
Eval, after cleaning	19	0.437	0.270	0.947	3 images excluded
Dev, before cleaning	88	0.457	0.421	0.41	many extra masks
Dev, after cleaning	87	0.481	0.519	0.816	1 image excluded

Reference mask versus ground truth

The SAM masks are model output. They are **not** manually annotated or manually verified ground truth. I looked at the contact sheets and removed the worst problems, but I did not outline eggs by hand. All my metrics therefore measure how closely my method agrees with SAM, not how correct it is. Where SAM is wrong, a perfect method would be penalised. The task asks for SAM output as the reference, so I used it, but I treat the numbers with that limitation in mind.

Task 2: Classical Image Processing Pipeline

I built the pipeline with help from an AI assistant (see Section 13), and then ran, tested and tuned it myself. The central idea is to separate two questions: (1) which regions could be an egg, and (2) which of them most looks like an egg. Question 1 is answered by simple thresholding on many channels, question 2 by a shape score. Table 2 lists every technique in the final method, and techniques I tried and dropped are listed in Section 6.

Table 2: Techniques in the final method.

Technique	What it does and why I used it	Parameters (final)	Limitation I saw
Resize + Gaussian blur	Brings every image to the same scale (long side 512 px) so that area limits and kernel sizes mean the same thing; blur removes sensor noise before thresholding.	long side 512, INTER_AREA; blur kernel 3 (picked by the grid search)	Blur alone gave almost no gain in the experiments (+0.003).
Colour channels	A pale egg may stand out in one channel only. I compute 7 maps: gray, Lab L, a, b, HSV saturation, value, and the distance to the median colour of a thin border ring (an estimate of the background).	gray, L, a, b, sat, val, bg-distance	Seven channels create many wrong candidates too.
Otsu thresholding	Chooses the threshold that best separates the histogram into two groups (maximises the between-class variance). Each channel gives a bright and a dark side, so I get 14 binary candidate maps.	2 classes, both polarities	One global threshold: a pale egg on pale cloth is often not separated.
K-means clustering	Clusters pixels in Lab colour space; each cluster is one more candidate map. Helps when the background has several colours.	k = 3, fixed random seed	Clusters follow big colour areas, not objects.
Morphological closing	Fills small holes and gaps (egg spots, glare) so that an egg stays in one piece.	elliptical kernel, 5 px	Cannot repair large missing parts.
Contours and connected regions	Every outer contour of every candidate map becomes a filled region. Regions that are far too small or too large are skipped before scoring.	area 1%-75% of the image	Touching eggs form one region.

Technique	What it does and why I used it	Parameters (final)	Limitation I saw
Shape-based scoring	Rates how egg-like a region is (Table 3). This is the part that does the actual selection.	weights 0.50 / 0.15 / 0.20 / 0.15	Rejects pale eggs whose edge is weak (score below 0.8).
Non-max suppression + threshold	Sorts regions by score, drops overlapping ones (IoU above 0.4), keeps the ones with score of at least 0.8, up to 3 eggs.	min score 0.8, max 3 eggs	If nothing reaches 0.8 the best region is still returned (see Section 11).
GrabCut	Refines the boundary using colour models of foreground and background. Starts from the chosen region (inner part sure foreground, a band around its probable background).	3 iterations, band 3% of the shorter side; rejected if IoU with the start is below 0.6	Slow, and can leak into similar colours.

Table 3: The terms of the shape score. Each term is scaled to the range 0 to 1 before weighting.

Term	How it is computed	Weight	Idea
Ellipse fit	IoU between the region and the ellipse fitted to its outline, rescaled as (IoU - 0.6) / 0.35	0.50	Eggs are oval; a rectangle fits an ellipse badly (about 0.65).
Convexity	Area divided by convex-hull area, rescaled as (x - 0.8) / 0.2	0.15	Eggs have no dents.
Edge strength	Mean Sobel gradient on the outline divided by the 95th percentile of the gradient in the image	0.20	A real egg has a visible edge.
Axis ratio	Minor / major axis of the fitted ellipse, rescaled as (x - 0.4) / 0.2	0.15	Eggs are about 0.65 to 0.9.
Border penalty	0.2 times the fraction of the outline lying on the image border	minus	Eggs are normally fully in the frame.
Skin penalty	0.5 times the fraction of skin-coloured pixels (YCrCb) inside the region	minus	Hands should not count as eggs.

The rescaling of the ellipse and convexity terms was added after a mistake: with the original un-scaled terms, a large rectangular table region scored almost as high as an egg in the video test, because every convex shape has convexity near 1. After rescaling, real eggs score around 1.0 and the rectangle falls to about 0.55.

Dataset Splitting and Leakage Prevention

I split the 110 images into a development set (88 images) and an evaluation set (22 images), a ratio of 4:1. The development set is used for all design decisions and tuning. The evaluation set is only used to report the final result. A 4:1 split keeps most of the small dataset for tuning, while still leaving some images that the method has never been tuned on.

Why leakage matters here. Roboflow exports often contain several augmented copies of the same photo (file names contain .rf. followed by a hash), and photos taken in bursts look almost identical. If one copy is in the development set and another in the evaluation set, the evaluation is too optimistic because the method has effectively seen the image already.

How I reduced it. split.py groups images before splitting. Two images go into the same group if (a) they have the same source name once the Roboflow suffix is removed, or (b) their perceptual hashes are close. The hash is a 64-bit difference hash (dHash): the image is converted to gray, resized to 9x8, and each bit says whether a pixel is brighter than its right neighbour. Two images are near-duplicates if their Hamming distance is at most 5. Groups are merged with a union-find structure. The groups, not single images, are then shuffled (seed 42) and assigned, largest first, to the development set until it holds 80% of the images.

Table 4: Dataset split (counts are from the output of split.py; used images are after excluding the four images with a bad reference mask).

	Development	Evaluation	Total
Images in the split	88	22	110
Images used in experiments / evaluation	87	19	106
Number of groups			109
Smallest hash distance between the two sets			12 (threshold 5)
Random seed / dHash threshold			42 / 5

There were 109 groups for 110 images, so only one pair of images was grouped together. This dataset therefore had little duplicate leakage to begin with, and the smallest distance between any development and evaluation image is 12 bits, well above the threshold of 5.

Why the evaluation set must not be used for tuning. If I choose settings because they score well on the evaluation images, the evaluation number is no longer an honest estimate for new images: it is a tuned number. In this project I tried to follow the rule, but I should be open about what happened. I ran the evaluation on the evaluation set several times: before cleaning the reference masks, after cleaning them, and again after the last round of experiments (the configuration did not change in the last round, so the numbers are identical). In the first run I looked at the worst evaluation cases, which showed me the problem with the reference masks (Figure 4); I used this only to clean the references and not to change the algorithm. All later algorithm ideas (Section 6, T11 to T14) came from the development-set failures.

Experimentation and Experiment Log

All experiments ran on the 87 usable development images, with SAM masks as reference, using experiments.py. The logic is simple and automatic: each trial takes the best configuration so far and changes one thing. The change is kept only if the development mIoU improves by at least 0.005. One trial (T3, shape-based selection) was marked as an "enabler" because it cannot help alone without good candidates; it was kept anyway (it also improved mIoU on its own, +0.024). After the first pass, the trials that were discarded are tested once more on top of the final configuration, because a change can become useful once other changes are in. At the end a grid search tunes four hyper-parameters (54 combinations).

Table 5: Experiment log (development set, 87 images). "Gain" is the change in mIoU against the best configuration at that moment.

Exp.	Modification	Parameters	Dev mIoU	Decision	Reason
T0	Baseline	gray channel, Otsu, bright side, largest blob	0.3196	Kept	Starting point. Takes the biggest bright blob, which is often the hand or the background.
T1	Gaussian blur	kernel 5	0.3229	Discarded	Gain +0.0033, below the 0.005 limit.
T2	Morphological open + close	open 5, close 7	0.3229	Discarded	Gain +0.0033, below the limit.
T3	Shape-score selection, both polarities	bright + dark; ellipse, convexity, edge, axis ratio	0.3432	Kept	+0.0236. Choosing the most egg-like region beats choosing the biggest one.
T4	3-class (multi-)Otsu	bands bright / mid / dark	0.3076	Discarded	-0.0356. More candidates, but more wrong ones. Retest later: 0.4155, still lower.
T5	More colour channels	gray, L, a, b, sat, val, bg-distance	0.3883	Kept	+0.0451, the biggest single gain. Some eggs only stand out in one channel.
T6	Skin penalty	0.5 x skin fraction (YCrCb)	0.4020	Kept	+0.0137. Hands are pushed down the ranking.

Exp.	Modification	Parameters	Dev mIoU	Decision	Reason
T7	K-means candidates	k = 3 in Lab	0.4216	Kept	+0.0196.
T8	GrabCut refinement	3 iterations, reject if IoU < 0.6	0.4383	Kept	+0.0167. Boundaries follow the egg more closely.
T9	Ellipse regularisation	replace mask by fitted ellipse if IoU > 0.85	0.4302	Discarded	-0.0081. Real eggs are not perfect ellipses. Retest: 0.4499 vs 0.4576.
T10	Up to 3 eggs per image	max eggs 3	0.4576	Kept	+0.0193. Several images contain more than one egg.
Retry	Discarded T1, T2, T4, T9 on top of T10	as above	0.4471 / 0.4277 / 0.4155 / 0.4499	Discarded	All below 0.4576.
T11	Quantile threshold sweep	thresholds at quantiles 0.2, 0.35, 0.5, 0.65, 0.8, 0.9 of each channel	0.4211	Discarded	-0.0365. Worked on a synthetic test, but hurt on real images (see text).
T12	Hard border rejection	reject if > 15% of outline on the border	0.4576	Discarded	No change at all (0.0000).
T13	Centre prior	weight 0.5	0.4424	Discarded	-0.0152.
T14	Colour-contrast term	weight 0.5, region vs 3% ring, Lab	0.4047	Discarded	-0.0529. The worst idea of the round.
T15	Grid search	blur {3,5,9}, close {5,9,15}, min score {0.5,0.65,0.8}, max eggs {1,3}	0.4814	Kept	+0.0238. Best: blur 3, close 5, min score 0.8, max eggs 3.

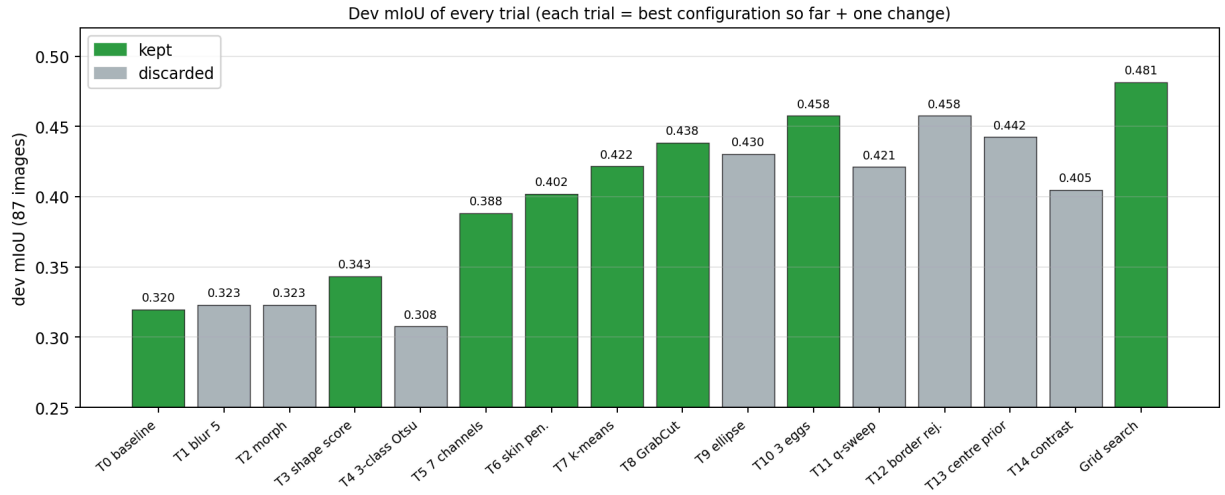


Figure 5: Development mIoU of every trial. Green bars were kept, grey bars were discarded (values from the experiment log).

What I learned from the log

- **Selection matters more than preprocessing.** The two preprocessing ideas (blur, morphology) gave almost nothing. The biggest gains came from choosing regions by shape (T3, +0.024) and from offering more candidates, i.e. more colour channels (T5, +0.045).
- **Several small, sensible ideas add up.** Skin penalty, k-means, GrabCut and allowing more eggs each gave between +0.014 and +0.020. The grid search gave a further +0.024. Together the log moves the development mIoU from 0.320 to 0.481.

- **Failures are informative.** The 3-class Otsu and the ellipse replacement made things worse, even when I retested them later. Ellipse replacement hurts because eggs are only roughly elliptical, and a hard replacement damages good masks.
- **Round 2 ideas did not work.** After looking at the development failures (Section 9), I tried four new ideas, T11 to T14. All were discarded. T12 gave exactly the same mIoU as before, which tells me that regions on the image border were not the real problem. T11 is the one I understand least: on a synthetic test with a pale soft-edged egg it raised IoU from 0.18 to 0.88, but on the real development images it lowered mIoU from 0.4576 to 0.4211. My guess is that it adds many extra candidates that are also egg-shaped, and with the default minimum score of 0.55 some of them win. I did not test it again together with the tuned minimum score of 0.8, so I cannot say it is useless.
- **The grid search ran into its own limits.** All four best values lie at the edge of the ranges I searched (smallest blur, smallest closing kernel, largest minimum score, largest egg count). The ranges were probably too narrow, and a wider search could find a better setting.

Final Segmentation Method

Figure 6 shows the final pipeline, and Table 6 the final configuration. The configuration is the result of the trials that were kept in the log (T0, T3, T5, T6, T7, T8, T10 and the grid search).

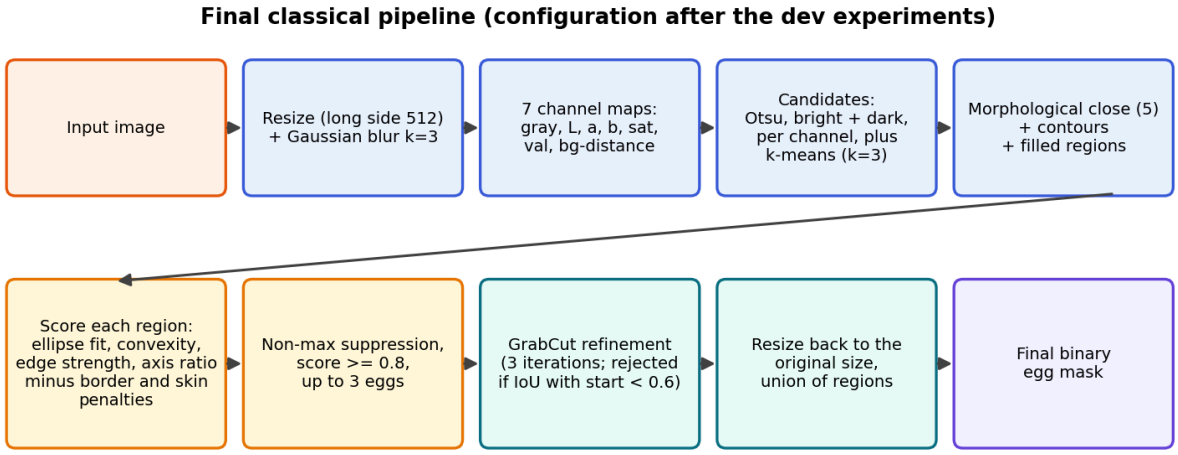


Figure 6: The final classical segmentation pipeline.

1. **Input and preprocessing.** The image is resized so that the long side is 512 px and blurred with a 3x3 kernel.
2. **Candidate generation.** For each of the 7 channels Otsu's threshold gives a bright and a dark binary map. K-means with 3 clusters in Lab gives 3 more maps.
3. **Clean-up.** Each map is closed with a 5 px elliptical kernel, outer contours are extracted and filled; regions outside 1% to 75% of the image area are skipped.
4. **Scoring.** Every region gets the shape score of Table 3 (ellipse fit, convexity, edge strength, axis ratio, minus border and skin penalties).
5. **Selection.** Regions are sorted by score; overlapping regions (IoU above 0.4) are removed; at most 3 regions with a score of at least 0.8 are kept. If none reaches 0.8, the best region is still returned.
6. **Refinement.** GrabCut (3 iterations) is run for each chosen region, the largest connected piece is kept and holes are filled. If GrabCut changes the region too much (IoU below 0.6) the original region is used.

7. **Output.** Masks are scaled back to the original image size; the union of all egg masks is the final binary mask.

Table 6: Final configuration, reconstructed from the experiment log (it is also saved by the code in results/final_config.json).

Parameter	Value	Comes from
channels	gray, L, a, b, sat, val, bg-distance	T5
polarities / Otsu classes	bright + dark / 2	T3 (T4 discarded)
selection	shape score	T3
skin penalty	0.5	T6
k-means	on, k = 3	T7
GrabCut	on, 3 iterations	T8
ellipse regularisation	off	T9 discarded
max eggs	3	T10 and grid
blur kernel	3	grid
closing kernel / opening	5 / off	grid / T2 discarded
minimum score	0.8	grid
border penalty / min area / max area / NMS IoU	0.2 / 1% / 75% / 0.4	defaults

This configuration was chosen over earlier ones simply because it has the highest development mIoU (0.4814) in the log. I did not choose it based on the evaluation set.

Task 3: Quantitative Evaluation

I compare the union of my predicted egg masks with the union of the SAM masks, pixel by pixel. The metrics are computed per image and then averaged. All of them come from evaluate.py.

$$IoU = \frac{|P \cap R|}{|P \cup R|}$$

$$Dice = \frac{2\,|P \cap R|}{|P| + |R|}$$

Here P is the predicted mask and R is the SAM reference mask.

- **IoU** is the overlap divided by the union. It is the main metric and the one I used for tuning. It is strict: a mask that is 10% too large and 10% shifted already loses a lot.
- **Dice** is similar but less harsh (it is always at least as high as IoU). I report it because many papers use it.
- **Precision** is the share of predicted pixels that are really egg (according to SAM). Low precision means the prediction leaks into the background or hand.
- **Recall** is the share of the reference egg that I found. Low recall means part of the egg is missing.
- **Boundary F1** checks only the outline: a predicted boundary pixel is correct if a reference boundary pixel is within 1% of the image diagonal, and the same in the other direction. It shows boundary quality independently of mask size.
- **HD95** is the 95th percentile of the distances between the two boundaries (the larger of the two directions), in pixels. It is a "worst places" measure and is large whenever a prediction lies somewhere else.
- **Count agreement** is the share of images where the number of predicted eggs equals the number of reference eggs.

Table 7: Development set (87 images; used for tuning, so these numbers are optimistic). HD95 in pixels.

Metric	Mean	Std	Median	Min	10th pct.
IoU	0.4814	0.3726	0.5186	0.0000	0.0000
Dice	0.5564	0.3811	0.6830	0.0000	0.0000
Precision	0.5926	0.4379	0.9171	0.0000	0.0000
Recall	0.6434	0.3860	0.7926	0.0000	0.0000
Boundary F1	0.4526	0.3444	0.4763	0.0000	0.0000
HD95 (px)	182.43	149.19	178.67	1.69	9.94

Table 8: Held-out evaluation set (19 images; final configuration; this is the number to report). HD95 in pixels.

Metric	Mean	Std	Median	Min	10th pct.
IoU	0.4367	0.3218	0.2698	0.0000	0.1085
Dice	0.5412	0.3190	0.4249	0.0000	0.1910
Precision	0.5190	0.4032	0.3199	0.0000	0.1137
Recall	0.7723	0.3144	0.9462	0.0000	0.3829
Boundary F1	0.3574	0.3062	0.2642	0.0000	0.0000
HD95 (px)	198.48	135.15	246.41	13.69	24.84

Count agreement is 0.816 on the development set and 0.947 on the evaluation set. The share of images with IoU above 0.8 is 0.276 (development) and 0.211 (evaluation).

How I read these numbers

- **The method is hit-or-miss.** The standard deviation of IoU is about 0.32 to 0.37, as large as the mean, and the median is far from the mean on the evaluation set (0.27 vs 0.44). Images are either segmented fairly well or not at all. A mean IoU of 0.44 is therefore not "a bit worse than perfect everywhere"; it is a mix of good results and complete misses.
- **The evaluation score is lower than the development score** (0.437 vs 0.481), as expected, because the development set was used for tuning. But the evaluation set has only 19 images. With a standard deviation of 0.32, the standard error of the mean is about 0.07, so the 95% interval is roughly plus or minus 0.15. I cannot claim that the gap between 0.437 and 0.481 is meaningful in either direction.
- **Precision and recall differ between the sets.** On the evaluation set the median recall is 0.95 but the median precision only 0.32: my masks usually cover the egg, but also much more (leaking into hand or background). On the development set it is the other way round (median precision 0.92, median recall 0.79). This is a sign of small-sample noise as much as of a real difference.
- **Boundary F1 is low (0.36 to 0.45)** and HD95 is large (180 to 200 px). The large HD95 is mostly caused by images where the predicted mask is somewhere else, not by sloppy boundaries.
- **Count agreement improved** from 0.41 to 0.82 on the development set after the reference masks were cleaned (Table 1), which shows how many "extra eggs" came from stray SAM masks.

Table 9: Failure categories (rules in evaluate.py: IoU of at least 0.8 is "ok"; IoU below 0.1 is "wrong object or miss"; recall of at least 0.85 with precision below 0.8 is "over-segmentation"; precision of at least 0.85 with recall below 0.8 is "under-segmentation"; everything else is "mixed").

Category	Development (87)	Evaluation (19)	Meaning
ok	24	4	good segmentation

Category	Development (87)	Evaluation (19)	Meaning
wrong object / miss	21	2	egg not found
under-segmentation	20	3	part of egg missing
over-segmentation	17	8	leaks into hand or background
mixed boundary error	5	2	both errors



Figure 7: Best development results. Columns: image, SAM reference (green), my result (red), error map (red: false positive, blue: false negative). The third row has a high IoU (0.99) but both SAM and my method segmented the whole lid with its label together with the egg, so a high IoU means agreement with SAM, not necessarily a correct egg.

Qualitative Analysis and Failure Cases

Figures 8 and 9 show the six worst development cases. All six have IoU of 0.00. The dominant failure on the development set is not a boundary problem; it is that the method returns something other than the egg. In 21 of 87 development images the method found the wrong object or nothing.

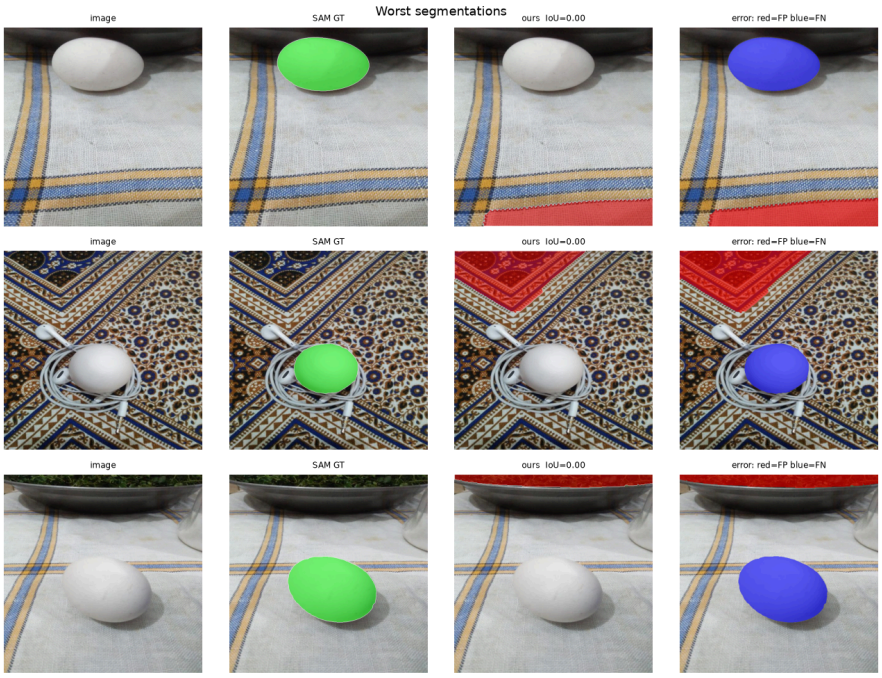


Figure 8: Worst development cases, rows 1 to 3. Columns: image, SAM reference (green), my result (red), error map (red: false positive, blue: false negative).

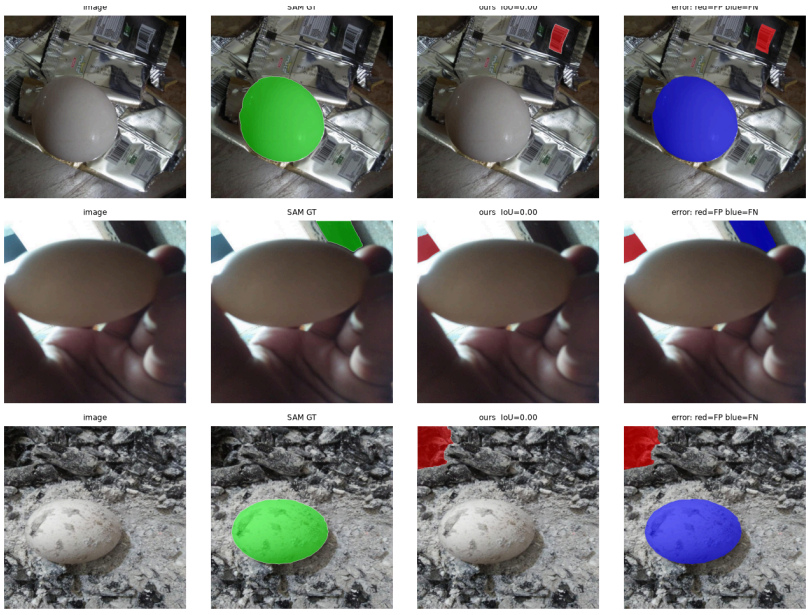


Figure 9: Worst development cases, rows 4 to 6.

Table 10: What happened in each of the six worst cases, and a possible improvement.

Row	What went wrong	Likely reason	Possible improvement
1	White egg on a cloth was not found. A red region in the bottom right corner was returned instead.	Pale egg on a pale cloth: weak edge, so the score is below 0.8; the best leftover region is returned anyway.	Return nothing (or flag it) when no region reaches the minimum score; add a candidate type that works on low contrast.
2	Egg with earphone cables missed; part of the carpet at the top left returned.	The cables cross the egg outline, and the carpet pattern creates many egg-like blobs.	Use a stronger prior (single object near the centre) or remove thin structures before scoring.
3	Egg next to a steel bowl missed; a strip along the bowl edge returned.	Low contrast between egg and cloth.	As row 1.
4	Egg on silver foil packets missed; a small red label returned.	Foil has strong reflections and many sharp edges; the egg edge is not special.	Learn what an egg looks like (a small classifier on regions) instead of using hand-made scores.
5	Close-up of a brown egg in a hand. SAM and my method both missed the egg.	The egg fills most of the frame, is skin-coloured, and my skin penalty works against it. SAM's own reference is a background piece here.	Do not penalise skin colour when the region is clearly an egg shape; better reference masks.
6	Egg on ash missed; a patch at the top left returned.	Textured, grainy background and a pale spotted egg.	Texture-aware preprocessing (stronger blur or median filter on the background).

9.1 What I could and could not show

The six worst cases are all of one category ("wrong object or miss"). The evaluation output also lists under-segmentation (20 development images), over-segmentation (17) and mixed boundary errors (5), and on the evaluation set over-segmentation is the largest category (8 of 19). I describe these categories with the metrics above, but I do not have a figure with examples of each category yet. Two further categories from the assignment, hand or skin detected as an egg and overlapping eggs, appear in the video part (Section 10) and in the earlier SAM examples (Figure 3), but I did not collect separate image examples from the evaluation output.

9.2 What the failures tell me

- The algorithm decides how egg-like a region is only from shape and edges. On pale eggs on pale backgrounds the edge term is weak, and the score falls below the 0.8 minimum.
- Because the code returns the best region even when it is below the minimum score (force_one), a failed detection becomes a confident wrong mask instead of an empty result. IoU is 0 in both cases, but precision and the image would look worse and I would not know the method is unsure.
- When I tried to fix this with better candidates (T11) or by removing border regions (T12), mean IoU did not improve. I do not yet know whether the egg is missing from the candidate list or is present with a low score. Checking this directly for the failing images is the next step I would take.

Task 4: Video Segmentation and Tracking

The video is a short educational clip (1440 frames at 24 fps, 60 seconds, 640 x 360 pixels) that shows two white eggs on a dark table. A hand moves and spins them. The eggs appear at about frame 223 and are on screen until about frame 1000; the rest of the video is title and text cards. The goal is to give each egg its own ID and keep it through the video.

First attempt and what was wrong

My first attempt used the image-trained configuration directly for detection, with a tracker that matched detections to tracks with the Hungarian algorithm on mask overlap and centroid distance, and dropped a track after 15 missed frames. The result had three clear problems (Figure 12, top row):

- The two eggs were **not detected for about 240 frames** while they sat still on the table.
- Overlays flickered on and off during hand movement.
- After the hand carried one egg, **the IDs were swapped**: the egg that stays on the left was labelled "egg 1" although it had been "egg 2".

Detector for the video

I checked the first problem frame by frame. In this video the eggs cover only about 1% of the frame, and the thresholds that worked for the dataset photos rejected them. The detector instead returned the dark reflections under the eggs, the logo and the caption text. For the video I therefore used a separate preset of the same method (Figure 11): only the brightness channel, a sweep over four fixed thresholds (150, 170, 190, 210), the same shape score, a minimum score of 0.85 and an area range of 0.3% to 3% of the frame. The logo area is blanked. On the clean frames the two eggs score about 1.0 while the false regions score 0.78 or less, so a threshold of 0.85 separates them. If a detected blob is much bigger than a typical egg (touching eggs), it is split with a watershed on the distance transform, or with k-means on the pixel coordinates as a fallback.

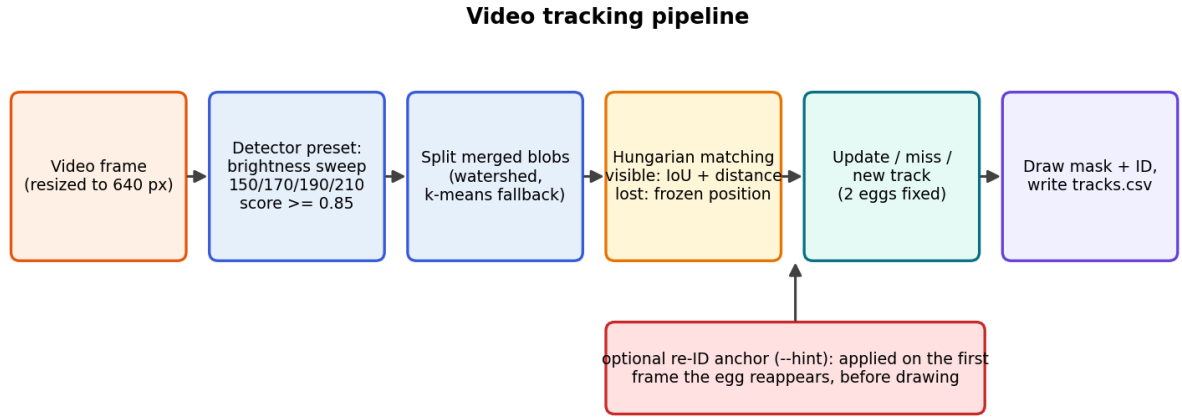


Figure 10: Video tracking pipeline.

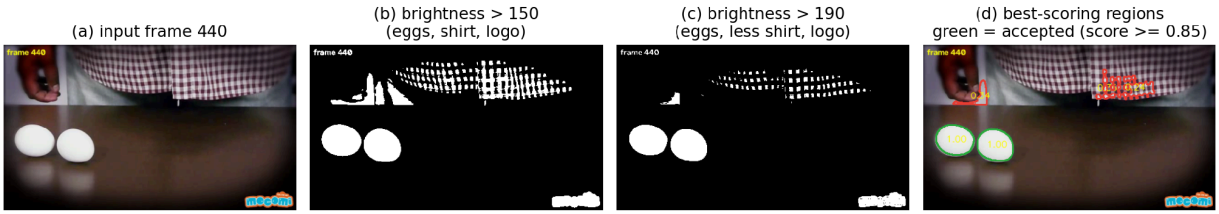


Figure 11: Detection in the video on a clean frame (frame 440). (b) and (c): brightness thresholds. (d) the best-scoring regions with their scores; the two eggs score 1.00 and are accepted (green), false regions score below 0.4 (red).

Tracker

The tracker is my own small implementation (not SORT or DeepSORT). Each frame, detections are matched to existing tracks with the Hungarian algorithm (scipy.optimize.linear_sum_assignment). The cost depends on the track state:

- **Visible tracks** (seen in the last two frames): cost is (1 - IoU between the velocity-shifted old mask and the detection) plus 0.5 times the centroid distance divided by a quarter of the frame diagonal.
- **Lost tracks** (hidden for more than two frames, for example under the hand): they stay frozen at the last position where they were seen, because an egg on a table does not drift. Their cost is 0.6 plus 0.4 times the normalised distance, so any detection can be matched, with the nearest one preferred.
- **Track creation and removal:** with two eggs fixed (--n_eggs 2) no new IDs are created after two tracks exist and lost tracks never expire, so an egg that is lifted and put down elsewhere gets its ID back. A track is only drawn after three confirmed frames. When a track is lost, its last outline is drawn thin for 8 frames.
- **Output:** the video with coloured masks and IDs, and a CSV file with the frame number, ID, centroid and area of every drawn track.

Results

Figure 12 compares three runs at the same frames. With the new detector (middle row) both eggs are found from the first frame they appear. But after the hand carries one egg, the left egg is still labelled "egg 1" (frame 756).

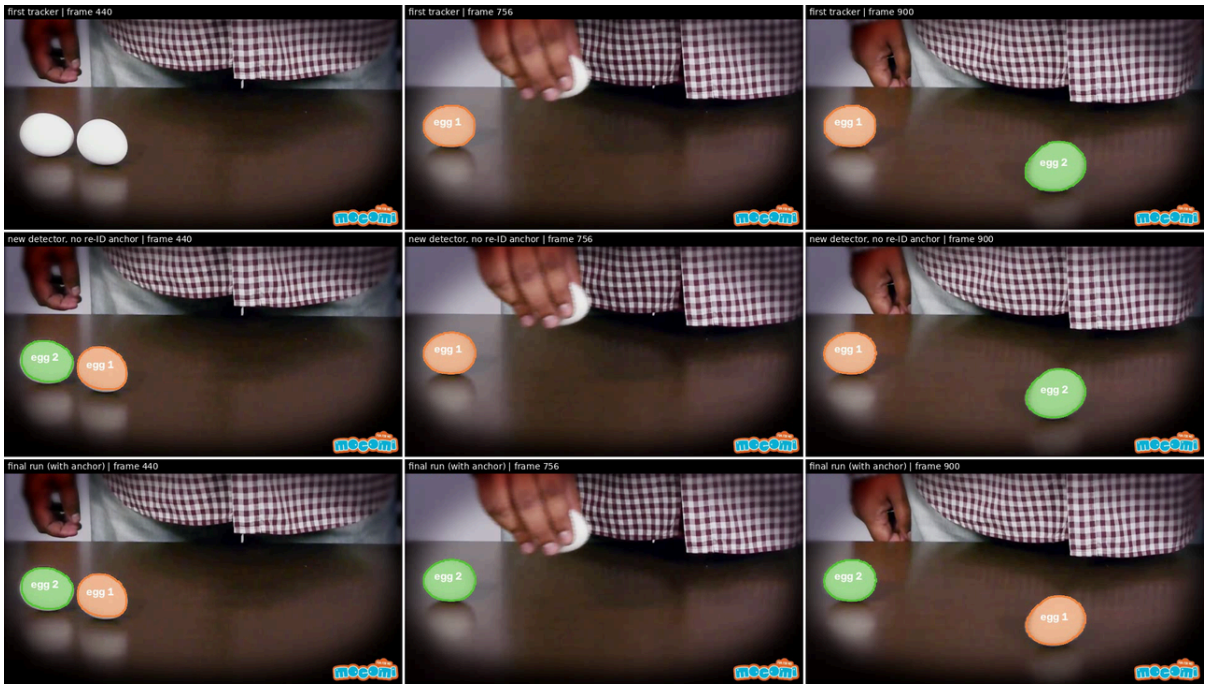


Figure 12: Same frames in three versions. Top: first tracker (eggs not detected at frame 440; IDs swapped at 756). Middle: new detector and tracker without a re-ID anchor (detected from the start, but still swapped at 756 and 900). Bottom: final run with one anchor.

Table 11: Number of frames (out of 1440) in which each egg is drawn as visible, from the summary line of each run.

Run	Egg 1	Egg 2	Note
First tracker (image config)	336	275	eggs missing for ~240 frames

Run	Egg 1	Egg 2	Note
New detector, no anchor	760	514	IDs swapped after frame ~750
New detector, anchor at frame 760	706	568	wrong label for ~10 frames, then fixed

The remaining swap is a real ambiguity, not just a bug. Around frame 740 the hand covers both eggs at the same moment and the two eggs have the same appearance. One egg was carried to the left earlier and was hidden while in the hand; the other was lifted from the table. When an egg reappears on the left, the position alone suggests the one that was last seen nearby, which is the wrong one. I could not find an automatic rule that gets this right, because the carried egg is only visible as weak fragments (scores 0.5 to 0.7) in the hand.

I therefore added a **re-ID anchor**: I told the tracker once that "from frame 745, the egg that appears within 60 px of (71, 193) is egg 2". The tracker applies this on the first frame where such an egg is visible, before anything is drawn. This is a manual correction and I want to be clear about it: the final video uses **one manual anchor** and is not fully automatic. With it, the final video keeps the IDs correct (Figure 13): egg 2 is carried from the left to the right and later back to the left, and egg 1 stays where it started until the hand carries it to the right side of the table.

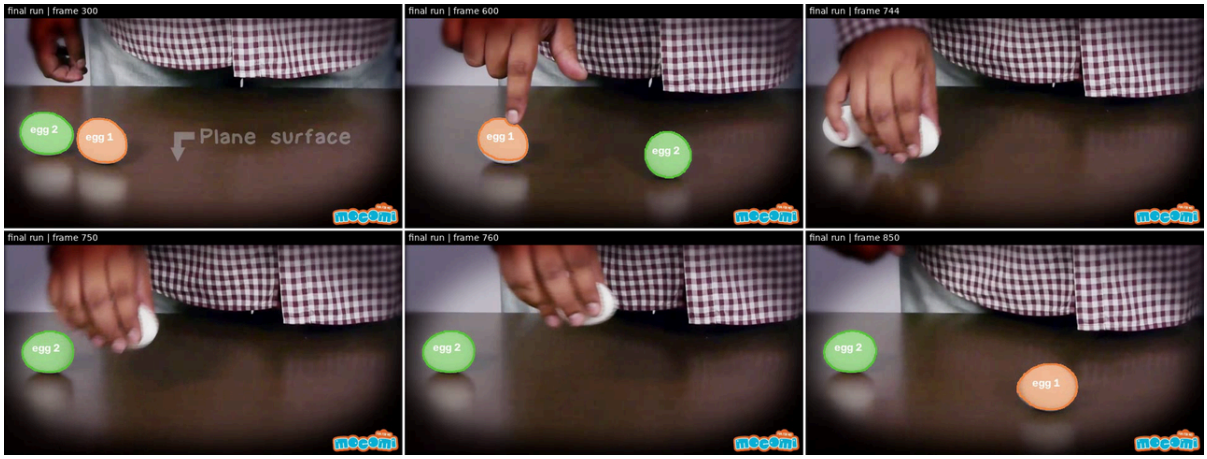


Figure 13: Final run: frames 300, 600, 744, 750, 760 and 850. Egg 2 (green) is the left egg at the start and is carried to the right (frame 600), returns to the left after the hand has hidden both eggs (frames 744 to 760); egg 1 (orange) is carried to the right (frame 850).

Video link:https://drive.google.com/file/d/1c9Dw0q2b2RSL1xPnScgAY0YC-cLooz40/view?usp=sharing

Challenges in the video

- **Identical eggs and occlusion.** The hardest part. Both eggs are white and the hand hides them, sometimes both at once, so identity cannot come from appearance.
- **Segmentation inconsistency.** Eggs held in the hand are partly hidden by fingers, and are detected only in some frames. The thin "lost" outline only partly hides the flicker.
- **Overlapping eggs.** In this video the eggs touch rarely. The watershed/k-means splitting works on a synthetic test with two touching eggs but I did not see a real touching case in this clip.
- **One video only.** The settings (thresholds, logo area, minimum score of 0.85) were chosen for this one video. I have not tested them on any other.

Challenges and Limitations

- **The reference is SAM, not ground truth.** All metrics measure agreement with SAM. I cleaned obvious mistakes, but other errors remain, and for some images (such as the lid in Figure 7) both methods give the same wrong answer.
- **Pale eggs on pale backgrounds.** The main failure of the method, as Section 9 shows. This was not fixed by the four ideas I tried in the second round.
- **Wrong output instead of no output.** force_one returns the best region even when it is below the minimum score. I did not test turning it off.
- **Hands and skin.** The skin penalty helped the average (+0.014) but also works against brown eggs held in a hand (Figure 9, row 5).
- **Parameter sensitivity and a narrow grid.** Every best value of the grid search was on the edge of the searched range, so the final setting may not be the real optimum. The minimum score of 0.8 also decides whether the egg is accepted at all.
- **Small evaluation set.** 19 images give a wide uncertainty (about plus or minus 0.15 on the mean IoU). I would not claim the method generalises from this.
- **Cost.** GrabCut and the many candidates make the method slow compared with a simple threshold, and my laptop restarted twice while running the second round of experiments. I measured the first version of the quantile sweep at about 700 MB of memory per image on a textured test picture, which I think was the cause, and fixed the way masks are allocated (it dropped to about 90 MB).
- **Tracking.** One manual anchor was needed. The detector and tracker settings are tuned to one video.
- **Evaluation hygiene.** I ran the evaluation set several times (Section 5). The algorithm was not changed because of those results, but a stricter protocol would run it only once.

12. Conclusion and Future Improvements

I built a complete pipeline: SAM reference masks, a group-based split, a classical segmentation method with a logged experiment history, an evaluation with seven metrics, and a video tracker. On the held-out set the classical method reaches a mean IoU of 0.437 against SAM, about 0.05 below the development score, and only 21% of the held-out images (28% of the development images) reach IoU 0.8. It does not match SAM closely, which is what the assignment asked me to try, and I would not call it good enough for real use.

What worked: choosing regions by an explicit shape score instead of by size, offering many colour channels as candidates, and adding the skin penalty, k-means and GrabCut. The log shows each step. What I learned: the evaluation reference matters as much as the method (cleaning the SAM masks changed the numbers more than several algorithm ideas), a trick that works on a synthetic test may fail on real images, and a failed experiment is worth writing down because it narrows down the cause. The video part taught me that tracking identical objects through occlusion cannot always be solved by the tracker alone.

Future improvements

- Find out, for each failing image, whether the egg is missing from the candidate list or only scored low (this decides whether to fix generation or scoring).
- Test turning off force_one, the quantile sweep together with the tuned minimum score, and a wider grid.
- Replace the hand-made score with a small learned classifier on region features, trained on the development set.

- Create manually verified reference masks for at least the evaluation images, and enlarge the dataset.
- Track with appearance cues that differ between eggs (size, marks) or with a Kalman filter, and detect "both eggs hidden" events automatically instead of using a manual anchor.
- Test the video detector on other videos.

AI tool disclosure

I used ChatGPT as a coding assistant to help with script development and debugging during this project.

My involvement went beyond just running the code. I worked on setting up the environment, carrying out experiments, checking the segmentation results and understanding where the approach was failing. While reviewing the SAM-generated masks, I noticed some incorrect annotations and decided which images should be excluded to make the evaluation more reliable.

I also compared the results from different experiments, looked into failure cases and explored ways to improve the segmentation. For the video-tracking part, I tested the implementation, went through the output frame by frame and identified issues such as incorrect ID associations. Whenever something did not work as expected, I looked into the issue, tried possible fixes and checked whether they improved the results.

ChatGPT helped me with coding and technical guidance, but I was actively involved in testing, analysing the results and making decisions throughout the project. I can explain the methodology, the results and the limitations of the work.